DATA CONTRACTS, SCHEMA EVOLUTION, AND BACKWARD COMPATIBILITY

Engineering Case Study for Enterprise Data, APIs, Messaging, Documents, and AI

Author	Venkata Lakshmi Parimala Pasupuleti
Primary domains	Affordable Housing, data engineering, backend systems, production AI
Systems represented	Income limits, eligibility, RabbitMQ, documents, migrations, LeaseI, AgentProbe
Document type	Technical engineering case study
Primary objective	Evolve shared contracts without breaking producers, consumers, historical behavior, or customers
Implementation boundary	Production experience plus clearly labeled target contract practices
Operating principle	A contract includes meaning, ownership, compatibility, and migration - not only field names

This document does not claim ownership of a company-wide schema registry or data-governance platform. It captures contract and compatibility practices grounded in database migrations, merge-field redesign, RabbitMQ workflows, annual data processing, APIs, and AI architecture.

Document Map

1. Executive Summary
2. Scope and Evidence Boundary
3. What Constitutes a Contract
4. Compatibility Models
5. Contract Ownership and Discovery
6. Database Schema Evolution
7. Stored Procedure and View Contracts
8. ETL and Source Data Contracts
9. API Contracts and Versioning
10. RabbitMQ Message Contracts
11. Document Merge-Field Contracts
12. Configuration Contracts
13. Historical and Effective-Dated Behavior
14. Expand-and-Contract Migrations
15. Backfill and Reconciliation
16. Testing Strategy
17. Release Sequencing and Consumer Coordination
18. Observability and Usage Evidence
19. Security, Privacy, and Tenant Isolation
20. AI Tool and Retrieval Contracts
21. Incident Examples and Lessons
22. Failure Modes and Trade-offs
23. Modernization Roadmap
24. Reusable Contract Checklists
25. Lessons Learned

1. Executive Summary

A data contract defines what one component promises to another: structure, meaning, ownership, allowed values, versioning, error behavior, and change rules. Contracts exist in database schemas, stored procedures, APIs, messages, document merge fields, configuration, files, and AI tools. A schema can remain syntactically valid while its meaning changes in a way that breaks customers.

My strongest contract-related experience came from an enterprise merge-field database redesign, production database migrations, annual income-limit transformations, RabbitMQ processing, AWS document integration, and long-term ownership of Affordable Housing workflows. These systems required old and new behavior to coexist while customers continued to generate documents, calculate eligibility, and process financial workflows.

The senior engineering responsibility is to identify every producer and consumer, classify compatibility, choose a migration pattern, preserve historical behavior, validate business outcomes, and remove old contracts only when usage evidence proves it is safe.

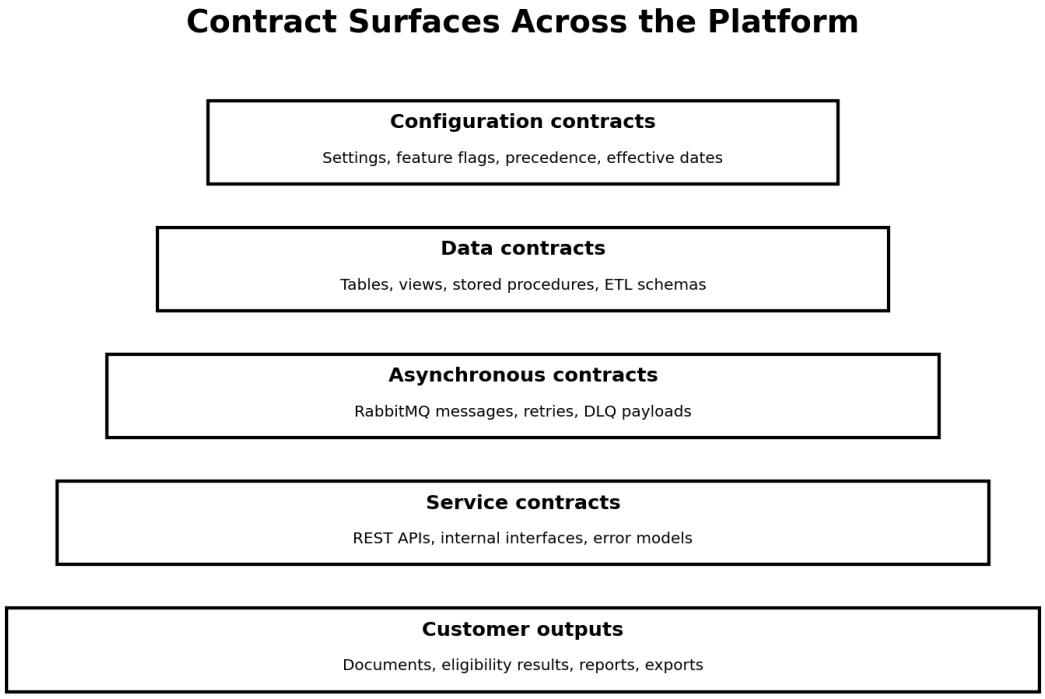


Figure 1. Contracts exist across configuration, data, messaging, services, and customer outputs.

2. Scope and Evidence Boundary

This case study combines direct production experience with target practices that strengthen contract governance. Direct experience includes owning the Affordable Housing side of a merge-field redesign, supporting database migrations, building data transformations, integrating asynchronous workflows, and maintaining business-critical SQL. Registry automation, formal enterprise compatibility gates, and broad platform governance are presented as modernization targets rather than completed company-wide implementations.

Area	Direct experience	Target maturity
Database	Schema changes, stored procedures, migrations, validation	Automated compatibility analysis and migration policy
Documents	Old-to-new merge-field mapping and urgent post-release defects	Versioned field registry with consumer usage
Messaging	RabbitMQ producers, consumers, retries, idempotency concepts	Schema registry and automated producer-consumer testing
APIs	Backend service integration and structured data access	Formal OpenAPI lifecycle and deprecation telemetry
AI	Python-owned tool access and structured investigation flow	Versioned tool schemas and evaluation gates

3. What Constitutes a Contract

Contract dimension	Questions
Structure	Which fields, types, keys, nullability, ordering, and nesting are valid?
Semantics	What does each value mean, including units, effective dates, and business interpretation?
Constraints	Which ranges, enums, uniqueness rules, and relationships are required?
Ownership	Who approves changes, answers questions, and supports incidents?
Version	How are versions identified and which combinations are supported?
Compatibility	Can old and new producers and consumers operate together?
Lifecycle	How are changes introduced, observed, deprecated, and retired?
Security	Which tenant, role, and data-classification rules apply?

Field names alone are insufficient. An income_limit value without program, geography, household size, effective year, currency, and source version is structurally present but semantically incomplete.

4. Compatibility Models

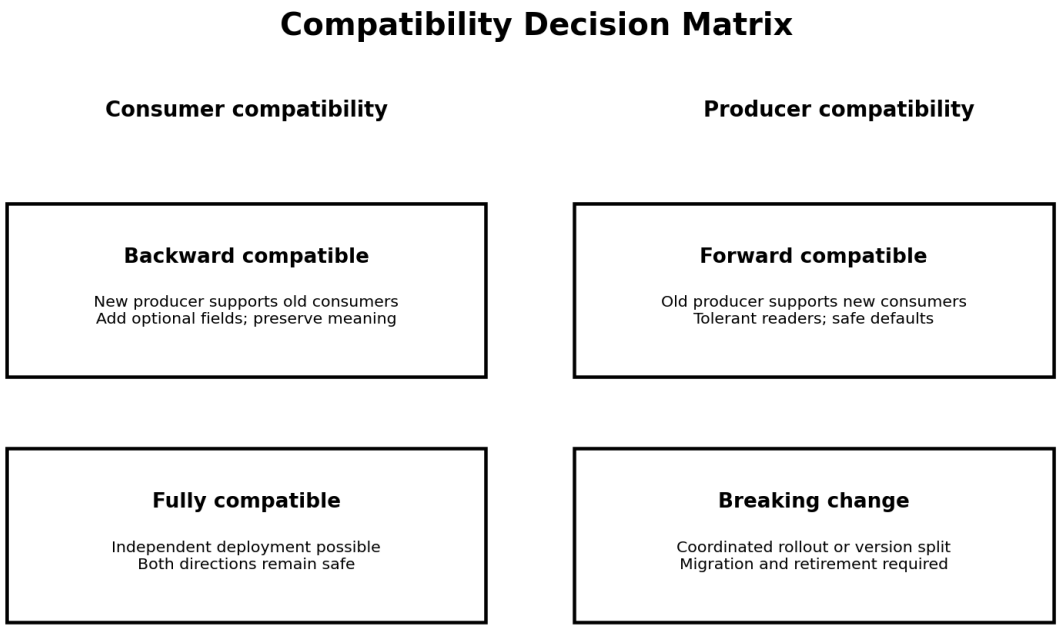


Figure 2. Compatibility direction determines whether independent deployment is safe.

Change	Typical classification	Reason
Add optional field with safe default	Backward compatible	Old consumers ignore it; new consumers handle absence
Add required field	Potentially breaking	Old producers cannot supply it
Rename or remove field	Breaking	Existing consumers still reference the old name
Widen allowed enum	Potentially breaking	Old consumers may reject unknown values
Narrow validation range	Breaking for previously valid data	Existing producers may send rejected values
Change unit or meaning	Breaking even if type is unchanged	Semantic behavior changes silently
Add new endpoint or message version	Compatible when old version remains	Consumers migrate independently

5. Contract Ownership and Discovery

The first challenge is often not designing the new schema; it is discovering hidden consumers. Database tables may be read by stored procedures, reports, documents, exports, support scripts, scheduled jobs, and other teams.

- Inventory direct and indirect producers and consumers.
- Identify business owner, technical owner, data steward, and operational owner.
- Search code, SQL, reports, message bindings, document templates, and support procedures.
- Classify criticality, tenant scope, data sensitivity, and historical dependencies.
- Record expected deployment independence and supported compatibility window.
- Create a retirement owner so deprecated contracts do not remain forever.

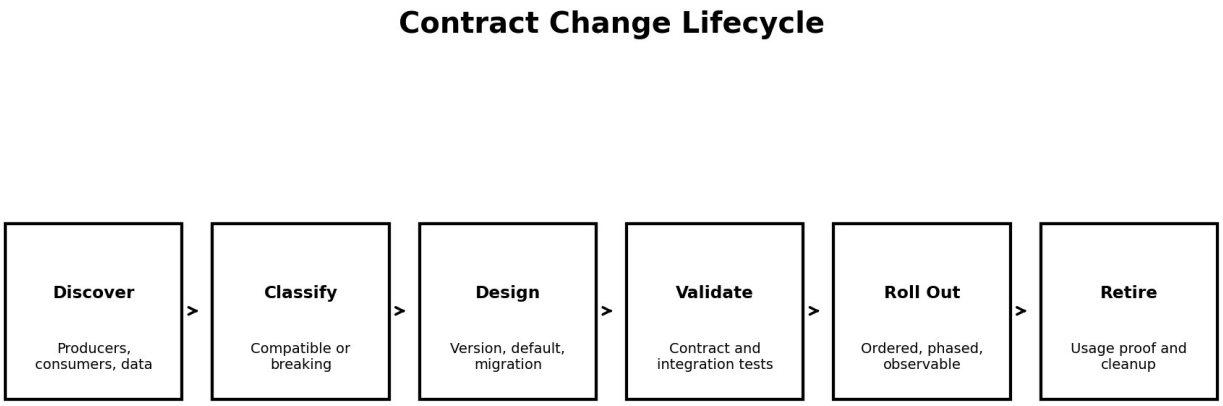


Figure 3. Contract changes move from discovery and classification through rollout, observation, and retirement.

6. Database Schema Evolution

Database evolution is difficult because schema and data state change together. Application code, stored procedures, reports, and migrations may deploy at different times.

Change type	Safer evolution approach
Add column	Add nullable or with safe default; deploy tolerant readers; backfill; enforce later
Rename column	Add new column, dual-write or synchronize, migrate readers, then remove old
Split table	Create new structure, backfill, dual-read or compatibility view, migrate consumers
Merge tables	Build canonical model and compatibility view; reconcile duplicates and ownership

Change type	Safer evolution approach
Change key	Introduce stable new key and mapping; avoid immediate destructive replacement
Change type	Add new representation, convert with validation, support both during transition
Drop object	Prove no usage through code search, telemetry, and owner approval

An atomic migration is attractive but often unrealistic across many consumers. Expand-and-contract allows mixed versions to operate during a controlled transition.

7. Stored Procedure and View Contracts

Stored procedures expose input parameters, output columns, side effects, transaction behavior, error semantics, and performance expectations. Views expose names and meanings that reports and application code may treat as stable.

- Version result sets when column order or meaning changes.
- Avoid changing a parameter from optional to required without a transition.
- Preserve error categories used by callers.
- Document transaction and idempotency expectations.
- Use compatibility views when physical schema changes before consumers.
- Compare execution plans and result sets for representative data.
- Retain historical effective-date behavior during refactoring.

8. ETL and Source Data Contracts

Annual income-limit processing depends on an external source contract and an internal publication contract. Source files or APIs can change column names, formats, codes, or completeness without warning.

ETL stage	Contract controls
Acquisition	Source identity, year, checksum, schema version, retrieval timestamp
Parsing	Explicit types, required columns, tolerant optional fields, rejected-row capture
Staging	Raw preservation, source lineage, duplicate business-key checks
Transformation	Versioned rules, expansion expectations, units, rounding, effective dates
Publication	Stable business keys, active version, atomic state transition
Consumption	Eligibility, rent, document, and report validation against representative cases

A few thousand source records expanding to roughly 180,000 published records creates a strong need for count, uniqueness, and semantic reconciliation. A technically successful load can still violate the internal contract.

9. API Contracts and Versioning

API dimension	Contract expectation
Request	Field types, required context, tenant scope, validation errors
Response	Stable meaning, optional extension fields, pagination, ordering
Errors	Machine-readable category, retryability, customer-safe message
Authorization	Role, tenant, property, and object-level access
Performance	Timeout and payload expectations where consumers depend on them
Versioning	Path, header, media type, or capability negotiation
Deprecation	Notice, migration guide, usage telemetry, retirement date

A new API version should exist only when compatibility cannot be preserved reasonably. Excessive versions create operational and support cost; silent breaking changes create customer risk.

10. RabbitMQ Message Contracts

Asynchronous contracts are especially sensitive because producers and consumers deploy independently, messages can remain in queues, and retries may occur after a new release.

Message element	Required design
Envelope	Message ID, type, version, timestamp, producer, correlation ID
Business key	Stable identity for idempotency and reconciliation
Payload	Minimal data with explicit units and optional-field handling
Version	Consumer-selectable or independently routable schema
Failure	Retry category and dead-letter metadata
Security	Tenant context and no unnecessary sensitive content
Retention	Compatibility for messages already in flight or delayed

- Consumers should ignore unknown optional fields when safe.
- Producers should not assume immediate consumer deployment.
- Enum growth requires tolerant handling or a version change.
- Acknowledgement and side-effect semantics are part of the contract.
- Replay behavior must preserve idempotency across versions.

11. Document Merge-Field Contracts

The enterprise merge-field redesign was a contract migration across templates, mappings, SQL, document-generation code, and historical customer documents. The field name was only one part of the contract; the source, formatting, null behavior, and version also mattered.

Merge-field concern	Compatibility requirement
Field identity	Stable identifier independent of display label
Data source	Documented query or domain source
Formatting	Date, currency, percentage, address, and empty-value behavior
Template version	Old templates continue to resolve during migration
Mapping	Explicit old-to-new mapping with validation
Retirement	No active templates or historical regeneration dependency
Observability	Missing field, deprecated usage, and generation failure telemetry

The six urgent defects in one day after the redesign demonstrated that migration completeness must be validated at the customer-output level, not only by confirming database mappings exist.

12. Configuration Contracts

Configuration is a contract between the system that defines a setting and every module that interprets it. Scope, type, default, precedence, effective date, and ownership must be explicit.

Configuration dimension	Example requirement
Scope	Global, tenant, property, module, program, user, or environment
Type and values	Boolean, enum, numeric range, structured object
Default	Safe behavior when value is absent
Precedence	Deterministic override resolution
Effective date	Historical and future activation behavior
Change event	Actor, reason, previous value, new value, release context

Configuration dimension	Example requirement
Consumer contract	Which modules depend on the setting and how they interpret it

The long eligibility investigation caused by another team's property setting illustrates a semantic contract failure: the setting existed, but its cross-module impact was not visible enough to the consuming team.

13. Historical and Effective-Dated Behavior

Regulated and financial systems must reproduce what was true at a prior date. A contract change cannot rewrite historical meaning unintentionally.

- Version rules, configuration, source data, templates, and mappings.
- Store effective-from and effective-to dates where the business meaning changes over time.
- Preserve the version used for a completed eligibility or document decision.
- Avoid recomputing historical outputs with current defaults.
- Define how corrections differ from new future-effective versions.
- Include time zone and observation-time semantics in migrations and audit evidence.

14. Expand-and-Contract Migrations

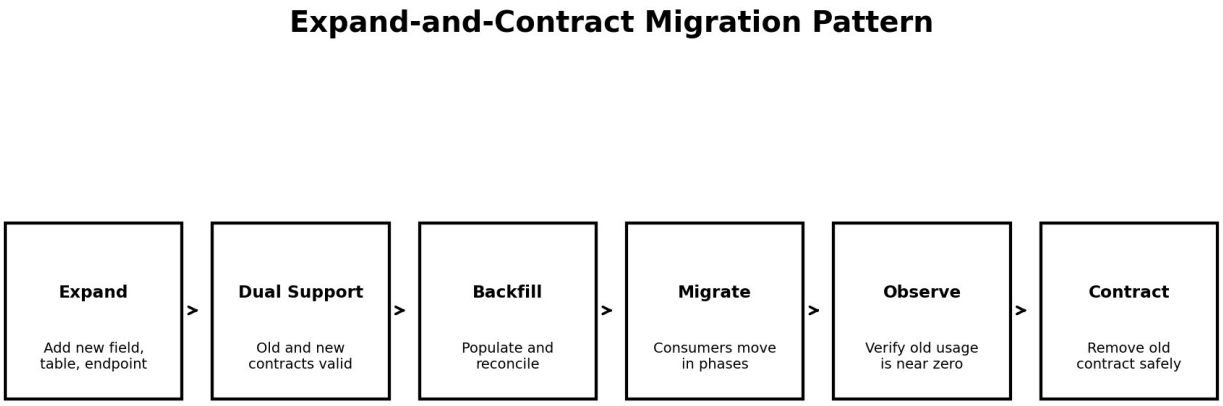


Figure 4. Expand-and-contract separates introduction, migration, observation, and removal.

The pattern reduces coordination risk by allowing old and new consumers to operate together. It is appropriate for database columns, tables, APIs, messages, and merge fields.

Phase	Engineering activities	Exit evidence
Expand	Add new contract without removing old; deploy tolerant readers	Mixed versions operate safely

Phase	Engineering activities	Exit evidence
Dual support	Write or expose both representations	No divergence beyond defined tolerance
Backfill	Convert historical data in bounded batches	Counts and business reconciliation pass
Migrate	Move consumers by priority and owner	All named consumers on new contract
Observe	Measure old usage, errors, and drift	Old usage reaches approved threshold
Contract	Remove old code, schema, mappings, and docs	Rollback and support plans updated

15. Backfill and Reconciliation

Backfills are production data changes and need the same discipline as migrations.

- Use bounded batches, checkpoints, and resumable execution.
- Record source version, transformation version, and affected scope.
- Make rerun behavior idempotent.
- Compare old and new representations before switching reads.
- Validate counts, nulls, duplicates, ranges, and representative business outcomes.
- Preserve exceptions for review rather than silently forcing conversion.
- Monitor database load, locks, queue impact, and customer workflows.

16. Testing Strategy

Test layer	Purpose
Schema validation	Reject malformed structures and invalid types
Producer contract test	Verify emitted data or message matches the declared contract
Consumer contract test	Verify required assumptions remain valid
Compatibility test	Compare old and new producer-consumer combinations
Migration test	Validate backfill, rerun, rollback, and mixed-version states
Integration test	Exercise database, API, queue, document, and configuration boundaries
Business regression	Confirm eligibility, rent, documents, and historical outcomes
Production validation	Observe representative real workflows and contract usage

Generated contract tests are valuable, but domain examples remain necessary. A schema can validate while a rent amount uses the wrong period or a merge field sources the wrong business entity.

17. Release Sequencing and Consumer Coordination

Deployment order depends on compatibility direction. A backward-compatible producer can usually deploy before consumers. A required new field may require consumers first or a versioned path.

Situation	Safer sequence
Optional response field	Producer first; consumers adopt later
Required producer input	Consumers or clients first with dual support
Database new column	Schema expand, tolerant application, backfill, switch reads, constrain later
Message version	New consumer support, new producer route, observe, retire old
Merge-field migration	Registry and dual mapping, template migration, usage proof, retirement
Breaking API	New endpoint/version, consumer migration, deprecation, removal

Release communication should name affected consumers, compatibility window, owner, validation, rollback, and retirement criteria.

18. Observability and Usage Evidence

Deprecated contracts should not be removed based only on code search. Runtime evidence helps find reports, scripts, old templates, delayed messages, and low-frequency consumers.

Signal	Use
Field or endpoint usage	Identify active consumers and migration progress
Message version rate	Measure producer and consumer rollout
Deprecated merge-field use	Find templates blocking retirement
Schema query telemetry	Detect direct database consumers
Compatibility errors	Reveal unknown fields, missing fields, or enum rejection
Dual-write divergence	Detect semantic mismatch during transition
Backfill progress	Estimate completion and validate checkpoints

19. Security, Privacy, and Tenant Isolation

- Contract changes must not weaken tenant filtering or object ownership.
- New fields require data-classification and access review.
- Message payloads should avoid unnecessary resident or document content.
- Compatibility layers must preserve authorization behavior.
- Backfills and migration scripts need bounded tenant scope and audit evidence.
- AI tool schemas must restrict operations and validate tenant context.
- Deprecated fields containing sensitive data should be removed from storage, logs, and documentation when policy allows.

20. AI Tool and Retrieval Contracts

LeaseI currently keeps API and database access inside Python functions rather than allowing the model to call systems directly. This creates a safer boundary, but production maturity requires explicit tool schemas, typed outputs, error categories, authorization, and versioning.

AI contract	Required control
Tool input	Typed parameters, allowed values, tenant context, validation
Tool output	Stable structure, source timestamp, freshness, error state
Knowledge document	Metadata, authority, effective date, access scope
Retrieval result	Document ID, score, source type, version, authorization
Agent state	Versioned fields and safe defaults across graph changes
Evaluation case	Input, expected behavior, evidence, failure category, baseline version

A model prompt change can also be a contract change when downstream code expects a specific JSON shape or confidence interpretation. The output must be schema-validated and regression-tested.

21. Incident Examples and Lessons

Incident or project	Contract lesson
Merge-field redesign	Mapping completeness and customer output must be validated together
Version 2 migration	Data state, timestamps, and observation context are part of migration evidence
Income-limit publication	External source and internal transformed schema both need validation

Incident or project	Contract lesson
RabbitMQ rent workflow	Message semantics include idempotency, acknowledgement, retry, and replay
Eligibility setting incident	Configuration meaning and cross-team dependency are contracts
AWS document migration	Shared framework contract and module-specific document semantics must align

22. Failure Modes and Trade-offs

Strict contracts versus evolution speed

Strict validation detects defects early but can block safe additive changes. Use compatibility policy and staged enforcement.

Versioning versus operational cost

A new version isolates breaking change but multiplies support, tests, and observability. Prefer compatible evolution when practical.

Dual writes versus divergence

Dual support enables migration but creates two representations that can disagree. Measure and reconcile continuously.

Direct database access versus agility

Direct access is fast initially but creates hidden consumers and retirement risk. Prefer governed interfaces for shared data.

Tolerant readers versus hidden defects

Ignoring unknown fields supports evolution, but silently accepting invalid meaning is dangerous. Tolerance must be bounded.

Long compatibility window versus technical debt

Long windows reduce consumer pressure but increase maintenance and confusion. Set owners and retirement criteria.

23. Modernization Roadmap

Phase	Focus	Outcome
1. Inventory	Catalog data, API, message, document, and configuration contracts	Known producers, consumers, and owners
2. Standardize	Define metadata, versioning, compatibility, and deprecation rules	Consistent change language

Phase	Focus	Outcome
3. Automate validation	Schema checks, contract tests, migration tests, CI gates	Earlier defect detection
4. Add usage telemetry	Observe versions, deprecated fields, endpoints, and templates	Evidence-based retirement
5. Govern migrations	Expand-and-contract templates, backfill controls, release sequencing	Safer mixed-version operation
6. Strengthen lineage	Source-to-output traceability and historical versions	Explainable business behavior
7. Extend to AI	Typed tools, versioned agent state, benchmark contracts	Safer production AI evolution

24. Reusable Contract Checklists

24.1 Contract change review

- Identify every producer and consumer.
- Document structural and semantic change.
- Classify backward, forward, full, or breaking compatibility.
- Define migration, rollback, ownership, and retirement.
- Add contract, integration, business, and security tests.
- Add usage and error telemetry.

24.2 Database evolution

- Expand schema without removing old paths.
- Deploy tolerant readers and writers.
- Backfill with checkpoints and reconciliation.
- Switch reads and writes in controlled phases.
- Prove no old usage before contraction.

24.3 Message evolution

- Version envelope and payload.
- Preserve in-flight message compatibility.
- Test old and new producer-consumer combinations.
- Define retry, DLQ, replay, and idempotency behavior.
- Observe version traffic before retirement.

24.4 Document-field migration

- Create stable field identifiers.
- Map old and new sources explicitly.
- Test active and historical templates.
- Monitor missing and deprecated fields.
- Retire only after template and regeneration review.

25. Lessons Learned

- A contract includes semantics, ownership, security, compatibility, and lifecycle - not just schema.
- Hidden consumers are often the largest migration risk.
- Effective dates and historical reproducibility are essential in regulated systems.
- Expand-and-contract reduces coordination risk but requires disciplined observation and retirement.
- Customer outputs such as documents are part of the compatibility surface.
- Asynchronous contracts must account for delayed messages, retries, and mixed deployment versions.
- Configuration and AI tool schemas deserve the same governance as APIs and database structures.
- Runtime usage evidence is stronger than assumptions when removing old contracts.
- Business regression testing is required even when structural schema tests pass.

Closing Perspective

Contract engineering is the discipline of changing systems without surprising the components, teams, or customers that depend on them. My experience with data transformations, database migrations, merge fields, RabbitMQ, documents, and AI reinforced one principle: compatibility must be designed, observed, and retired deliberately.